

APPLIED AGENTIC AI FOUNDATION PROGRAMME

Beginner Track · Windows · Terminal-first

MONTH 4 — LAB GUIDE

Build a Smart Recommender

An agent that recommends movies, songs, or books — by understanding similarity.

Builds on Months 1–3. Your first real taste of the math behind AI — learned by building, not lectures.

What you will build this month

- An agent that turns items (movies, songs...) into lists of numbers called vectors.
- It measures how similar any two items are — the core idea behind every recommendation engine.
- Given something you like, it finds and ranks the most similar items to suggest.
- It works on a real dataset and explains WHY it recommended each item — shipped as v1.0.

A special note about the “math” this month

- This month introduces **vectors**, the **dot product**, and **cosine similarity** — the first real AI math.
- Do NOT be intimidated. You will learn each idea by building it in code and seeing it work — never as abstract theory.
- If a concept feels fuzzy, that's fine. Run the code, watch the result, and the idea will click. Understanding follows doing.

Month 4 at a glance

How does Netflix suggest a film, or Spotify a song? The secret is simpler than you'd think: turn everything into numbers, then find the items whose numbers are “closest”. This month you build exactly that. Along the way you meet the foundational math of modern AI — but every piece of it is learned by making a working recommender.

Week	Focus	What you ship
Week 1	Vectors: turning things into numbers	A program that turns items into number lists (v0.1)
Week 2	Similarity: how “close” two items are	A tool that scores how similar two items are (v0.2)
Week 3	Recommending: find the closest matches	An agent that recommends the top matches (v0.3)
Week 4	Real data, cosine similarity & ship	A polished recommender on a real dataset (v1.0)

The big idea in one sentence: if we describe each item as a list of numbers, then “similar items” are just the ones whose number-lists are close together. That's the whole trick behind recommendations — and, later, behind how AI understands meaning.

Week 1 — Vectors: turning things into numbers

Goal of the week: learn what a vector is (just a list of numbers), and describe real items — like movies — as vectors your program can compare.

Day 1 — What is a vector?

A vector is nothing scary — it's just a list of numbers that describes something. You've already used lists in Month 2; a vector is a list of numbers with a purpose.

Task 1.1 — Describe a movie with numbers

1. Set up a new folder `recommender` and open it in VS Code. Create `day1.py`:

```
# Rate each movie 0-10 on: [action, romance, comedy]
die_hard    = [9, 1, 2]
notebook    = [1, 9, 2]
deadpool    = [8, 3, 8]

print("Die Hard as numbers:", die_hard)
```

2. Save and run.

✓ What you should see

Die Hard as numbers: [9, 1, 2]

That list [9, 1, 2] is a vector. It says Die Hard is very action (9), barely romance (1), barely comedy (2). Every item becomes a vector describing its “ingredients”.

🚩 **Checkpoint:** You understand a vector is just a list of numbers describing an item's features.

Day 2 — Vectors with NumPy

Plain lists work, but `numpy` (from Month 3) makes vector math easy and fast. Today you store vectors the NumPy way.

Task 2.1 — Make NumPy vectors

3. If NumPy isn't installed yet, run `pip install numpy` in the terminal. Then `day2.py`:

```
import numpy as np

die_hard = np.array([9, 1, 2])
notebook = np.array([1, 9, 2])
print(die_hard)
print("Add them:", die_hard + notebook)
```

4. Save and run.

✓ What you should see

[9 1 2]
Add them: [10 10 4]

`np.array(...)` turns a list into a NumPy vector. Notice adding two vectors adds them position-by-

position — NumPy does the whole list at once, no loop needed.

 **Checkpoint:** You can create vectors with NumPy and do math on them.

Day 3 — Multiply and sum (building toward similarity)

Similarity will use two operations: multiplying vectors position-by-position, then summing the result. Let's meet them now.

Task 3.1 — Multiply then sum

5. New file `day3.py`:

```
import numpy as np
a = np.array([9, 1, 2])
b = np.array([8, 3, 8])

multiplied = a * b
print("Multiplied:", multiplied)
print("Summed:", multiplied.sum())
```

✓ **What you should see**

Multiplied: [72 3 16]

Summed: 91

Multiply position-by-position ($9*8$, $1*3$, $2*8$), then add them up. That single number is a first measure of how much two items “agree”. You just did the core of the dot product.

 **Checkpoint:** You can multiply two vectors and sum the result — the building block of similarity.

Day 4 — Store many items together

A recommender needs a whole library of items. Store them in a dictionary (from Month 2): the name is the key, the vector is the value.

Task 4.1 — Build a movie library

6. New file `day4.py`:

```
import numpy as np

movies = {
    "Die Hard": np.array([9, 1, 2]),
    "Notebook": np.array([1, 9, 2]),
    "Deadpool": np.array([8, 3, 8]),
    "Toy Story": np.array([3, 4, 9]),
}

for title in movies:
    print(title, "->", movies[title])
```

✓ **What you should see**

```
Die Hard -> [9 1 2]
Notebook -> [1 9 2]
Deadpool -> [8 3 8]
Toy Story -> [3 4 9]
```

🚩 **Checkpoint:** You have a library of items, each stored as a named vector — ready to compare.

Days 5–7 — Combine & ship v0.1

- Day 5: Add at least 6 movies with your own honest ratings. Print the library neatly.
- Day 6: Review a new pitfall — vectors must be the **same length** to compare (all rated on the same features). If one has 3 numbers and another 4, math will fail. Keep them consistent.
- Day 7: Build v0.1 of `recommender.py`: the movie library plus a friendly welcome that lists all items. Run it start to finish.

🚩 **Checkpoint:** 🎉 Week 1 done. You can describe items as vectors and store a library. Next week: measuring similarity.

Week 2 — Similarity: how “close” two items are

Goal of the week: turn “multiply and sum” into a real similarity score, and build a function that tells you how alike any two items are.

Day 8 — The dot product

What you did on Day 3 — multiply position-by-position then sum — has a name: the dot product. A bigger dot product means the two vectors “agree” more.

Task 8.1 — Dot product the easy way

7. NumPy has this built in. New file `day8.py`:

```
import numpy as np
a = np.array([9, 1, 2])    # Die Hard
b = np.array([8, 3, 8])    # Deadpool
c = np.array([1, 9, 2])    # Notebook

print("Die Hard . Deadpool:", np.dot(a, b))
print("Die Hard . Notebook:", np.dot(a, c))
```

✓ What you should see

```
Die Hard . Deadpool: 91
Die Hard . Notebook: 20
```

`np.dot(a, b)` does the multiply-and-sum in one step. Die Hard scores higher with Deadpool (both action) than with the Notebook (romance) — the numbers already capture “similar taste”.

 **Checkpoint:** You can compute the dot product of two item vectors and read it as an agreement score.

Day 9 — A similarity function

Wrap it in a function (from Month 1) so you can reuse it everywhere.

Task 9.1 — Write `similarity(a, b)`

8. In `recommender.py`, near the top:

```
import numpy as np

def similarity(a, b):
    return np.dot(a, b)
```

9. Test it below your movie library:

```
print(similarity(movies["Die Hard"], movies["Deadpool"]))
```

✓ What you should see

```
91
```

 **Checkpoint:** You have a reusable similarity function — the engine of your recommender.

Day 10 — Compare one item to everything

To recommend, you compare your chosen item against every other item and see which score highest.

Task 10.1 — Score against the whole library

10. Add a loop that compares Die Hard to every movie:

```
chosen = "Die Hard"
for title in movies:
    score = similarity(movies[chosen], movies[title])
    print(title, "similarity:", score)
```

✓ What you should see

```
Die Hard similarity: 86
Notebook similarity: 20
Deadpool similarity: 91
Toy Story similarity: 44
```

Notice Die Hard scores highest with itself. That makes sense (it's identical!) but you don't want to recommend the same item — you'll skip it next week.

🚩 **Checkpoint:** You can score one item against every other item in the library.

Days 11-14 — Sort, present & ship v0.2

- Day 11: Collect the scores into a list of (title, score) pairs instead of just printing them.
- Day 12: Sort that list from highest to lowest score using Python's `sorted(...)` with a key. Introduce sorting gently.
- Day 13: Present results as a friendly ranked list, skipping the chosen item itself.
- Day 14: Ship v0.2 — a tool that, given an item, lists everything ranked by similarity.

🚩 **Checkpoint:** End of Week 2: you can rank the whole library by how similar it is to a chosen item.

Week 3 — Recommending: the top matches

Goal of the week: turn the ranked list into a real recommender agent — the user picks something they like, and it suggests the best matches with reasons.

Day 15 — The recommend function

Task 15.1 — Return the top 3 matches

11. Add a function that recommends the best matches for a chosen title:

```
def recommend(chosen, how_many=3):
    scores = []
    for title in movies:
        if title == chosen:
            continue # skip the item itself
        s = similarity(movies[chosen], movies[title])
        scores.append((title, s))
    scores.sort(key=lambda pair: pair[1], reverse=True)
    return scores[:how_many]
```

12. Test it:

```
print(recommend("Die Hard"))
```

✓ What you should see

```
[('Deadpool', 91), ('Toy Story', 44), ('Notebook', 20)]
```

`continue` skips the chosen item so it isn't recommended to itself. `scores[:3]` keeps the top three. `sort(..., reverse=True)` puts the highest first.

 **Checkpoint:** You have a `recommend()` function that returns the top matches for any item.

Day 16 — Make it a conversation

Task 16.1 — Let the user choose

13. Add a loop (like your Month 1 chat loop) that asks what they like and recommends:

```
print("Movies I know:", list(movies.keys()))
while True:
    choice = input("Tell me a movie you like (or bye): ")
    if choice == "bye":
        break
    if choice not in movies:
        print("I don't know that one. Try one from the list.")
        continue
    print("Because you liked", choice, "you might enjoy:")
    for title, score in recommend(choice):
        print(" -", title)
```

14. Save and run. Ask it for recommendations based on different movies.

 **Checkpoint:** Your recommender is now an interactive agent the user can talk to.

Day 17 — Explain WHY

- Add a short reason to each recommendation, e.g. show the similarity score or the shared strongest feature ("both are high on action").
- A recommender that explains itself feels far smarter and more trustworthy.

Days 18-21 — Polish & ship v0.3

- Day 18: Handle messy input — lowercase the user's choice and match it even with different capitalisation (your Month 2 trick).
- Day 19: Let the user say how many recommendations they want.
- Day 20: Add more items so recommendations feel richer.
- Day 21: Commit and push to GitHub. This is v0.3.



Checkpoint: A friendly, explaining recommender agent, live on GitHub.

Week 4 — Real data, cosine similarity & ship v1.0

Goal of the week: use a real dataset, upgrade to cosine similarity (fairer than the plain dot product), and ship version 1.0.

Day 22 — Why the dot product can be unfair

The plain dot product favours items with big numbers. An item rated [10,10,10] beats [5,5,5] on everything, even if they have the exact same taste profile. Cosine similarity fixes this by comparing direction, not size.

Task 22.1 — See the problem

15. Try this to feel the issue:

```
import numpy as np
small = np.array([1, 1, 1])
big   = np.array([10, 10, 10])
same_taste = np.array([2, 2, 2])
print("dot(small, same):", np.dot(small, same_taste))
print("dot(big, same):", np.dot(big, same_taste))
```

✓ What you should see

```
dot(small, same): 6
dot(big, same): 60
```

Both “same”, “small”, and “big” point the same direction (equal taste), yet big scores far higher just for having larger numbers. Cosine similarity removes that unfair size advantage.

Day 23 — Cosine similarity

Task 23.1 — Add a fair similarity function

16. Cosine similarity = dot product divided by the lengths of both vectors. NumPy makes it short:

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

17. Swap your `similarity` calls to use `cosine_similarity` and re-test. Scores now range 0 to 1, where 1 means identical taste.

`np.linalg.norm(a)` gives the length of a vector. Dividing by both lengths cancels out size, leaving only direction — a fairer match.

 **Checkpoint:** Your recommender uses cosine similarity, giving fair scores between 0 and 1.

Day 24 — Use a real dataset

- Load a real CSV of items (from Month 3 skills) — e.g. movies with numeric features, or songs with attributes — using `pandas`.
- Turn each row into a vector automatically, so your recommender works on hundreds of items you didn't type by hand.

Day 25 — Make it robust & friendly

- Handle unknown items, empty input, and datasets with different columns. Show helpful messages instead of crashing.

Days 26–30 — Final polish & ship v1.0

- Day 26: Test on a real dataset. Do the recommendations feel sensible? Tune your features if not.
- Day 27: Write a **README.md** explaining what it recommends and how similarity works, in your own words.
- Day 28: Record a short demo — pick an item, show the top recommendations and why.
- Day 29: Final commit and push (`git add .` → `git commit -m "v1.0"` → `git push`).
- Day 30: Show it off. You built a recommendation engine — the same core idea behind Netflix and Spotify.

Month 4 complete

This was a milestone month. You met the real math behind AI — vectors, the dot product, and cosine similarity — and you learned it the only way that truly sticks: by building something real with it. The idea that “meaning can be numbers, and similar things sit close together” is the exact foundation of how modern AI understands language, which you'll use directly when your agents gain memory later in the year. Month 5 builds on this to create an agent that reads text and judges its mood — your first classifier.